

单泓天实习心得总结

一、DL/T 2811 协议实现项目

1.1 项目背景

在来到公司实习一段时间学习了一些组里常用的技术栈后，我收到了第一个任务，参考国外的iec61850协议的实现，实现国产标准DL/T 2811。在接到任务后我先去查阅了一些电网相关的背景知识，逐渐了解了该协议提出的背景，使用的功能和预计达到的效果。DL/T 2811 ([CMS](#), [Communication Message Specification](#)) 是国家电网为了摆脱对国外 IEC 61850 [MMS](#) 协议的依赖，自主研发的电力通信报文规范。它的核心使命是实现电力通信的[自主可控](#)。🔗 [笔记](#)

1.2 实现过程

最开始我和同事与领导沟通过后，希望先找一下网络上公开的实现。这一阶段我一边摸索协议具体的细节，一边调研网络上相关的公开代码。最初的构想是找到iec61850协议与DL/T2811的区别，通过修改[ie61850](#)实现的底层编码等细节来完成协议实现的迁移。通过调研一些公开的工具发现现有工具很难满足要求，比如[asn1c](#)实现很老，并且不支持per编码，[liboss](#)实现支持完整的编码格式，但是需要付费。最后决定自己写一套基础的asn1文件per编码与解码的工具。

在实现完底层编码后，由于dlt2811相关内容的通信协议层比iec61850进行了简化，导致开源代码在体系上也有所区别。于是我以自己写的编码工具为基础逐层向上搭建，完成了基础数据结构（第七章），基础报文结构（第八章），网络服务器策略（第六章），scd文档解析，国密安全配置，一部分的客户端与服务器处理逻辑（8.2-8.5等）以及对应的命令行工具。

在和前辈沟通过后，他更建议底层的编码使用c/c++来开发，因为部署在的上位机可能性能比较差，并不能支持内存占用较高的JVM。所以我将数据结构与报文的数据结构与编码解码单独拿出来写成C语言的模块，并打包成dll，方便后续服务器或者客户端使用其他的技术体系开发。本实现的上层服务器等使用JNA映射沟通java与c。最后实现了目前的效果，下边开启服务器与客户端后，客户端命令行的效果：

```
bash

cms> # 连接服务器
cms> connect --ap C_B5041X/S1;
    Connecting to 127.0.0.1:8102 ...
    Connected, negotiating parameters ...
```

```
Negotiated, associating with C_B5041X/S1 ...
OK Associated: C_B5041X/S1
cms> # 请求某一访问点下支持的逻辑设备
cms> server-dir;
Logical Devices:
  [0] LD0
  [1] MEAS
  [2] CTRL
```

1.3 心得体会

1. ai辅助编程适合新项目，但是他更像一个放大器，在提高效率的同时也会扩大自身的缺点。比如当我对一些协议的细节并不了解的时候很容易会被ai提出的自认为正确的实现逻辑带跑偏。并且随着代码量的迅速提高，ai很容易把实现本身当成协议，忘记了这些实现只是去拟合协议的结果。为了解决这个问题，我将pdf的协议文档ocr成方便ai可以阅读的markdown文档，让ai有内容可以参考，而不是每次都要靠背景知识脑补。
2. ai辅助变成如何控制上下文。在领导的指导下，我了解了ai token计费的大部分其实是上下文的背景知识，而并不是输出。在后续的工作中，我将一些解析文档或者简单的修改交给网页窗口的ai，把需要大范围修改代码的工作交给编辑器的ai。另外随着trae的更新，新增了上下文手动压缩以及创建会话副本的功能。在进行大范围的修改前，可以先进行小范围尝试，当ai学会了这个模式以后，就创建会话的副本，防止大范围修改过程中上下文被填满被动触发压缩导致模型忘记需要做的事。
3. 不要过度设计。在一开始设计数据结构的过程中，我希望在支持了编码解码后，相同类型的数据结构”合并“到一起，在这个过程中，创建了很多抽象类，接口，钩子函数，组合与调用的关系，反而导致了维护成本的提高，开发效率的降低，经常不得不要推翻重做。在后期使用c重写数据结构的时候我注意让自己写的内容保持简洁，这样对ai理解我的代码也比较友善。
4. 持续学习。在使用c进行编码模块重构的时候，如何使用java调用c尝试了多种方法，一开始使用JNI，但每写一个函数都要写很多胶水函数，维护成本太高，后来换成了JNA及时止损。

二、基于场磨式直流电场传感器波形分析

2.1 项目背景

在前几周收到了另一个任务，帮助完成场磨式直流电场传感器的波形分析。具体内容是通过同时采集的一些电压波形数据，估计被观测的直流导线电压。

2.2 实现过程

在刚开展工作的几天里，我先阅读了领导给的论文，了解相关背景。接下来和同事一起分别设计模型与算法，通过电压数据进行估计。我设计了一个工具包，涵盖了查看数据库，从数据库下载数据，分析数据，清洗数据，训练模型，测试模型等功能。

在最初的几天，电压数据比较少（同一个电压数据很多，但不同电压本身比较少）。我在实现了几种传统的线性模型与树状模型后打算尝试神经网络，于是训练了一个参数比较少的CNN（LeNet），但由于数据较少发生了过拟合。

接下来的几天，同事更换了更多的电压，产生了更多的数据，每次有新的数据，模型就会更好一点。在这个过程中我尝试了更多的实验来分析并改进数据。比较成功的几个发现是：

1. 单帧 FFT 噪声大：从直觉上来讲，相近时刻的电压既然相同，那么这个电压产生的场磨信号电压得到的特征也应该相同，但是我按照时间顺序将这些特征排列出发现其波动很大。这是因为每一条测量数据只包含512个点，在计算fft的时候会含有噪声。解决方法是我设计了一个类似滑动窗口的结构，将相邻的n帧合并到一起计算fft（比如30帧，50帧）这样操作后输出波形稳定了很多，模型预测准确度也得到了改善。
2. 离散电压导致树模型“猜电压”：由于电压是离散的点，导致各种树的模型以及神经网络模型这种非线性的算法更倾向于猜电压，比如30V，50V。当真实结果是从没见过的48V。模型不认为这种会有这种没见过的电压的存在。另一方面，测量电压时会对某一个电压检测很长时间，导致电压和当时的温湿度绑定到一起了，模型可能看到了某一个温湿度，就知道这是那段时间测得电压了，从而猜出正确答案。为了解决这个问题，我尝试让模型更加“平滑”所以回退到普通的线性模型，然后使用温湿度进行修正。这个方法让模型预测准确度得到了很大的改善。

2.3 心得体会

1. 学习新知识要深度到原理，而不是浮于表面。在研究论文后和前辈讨论，我认识到的仅仅是对电压数据设计模型进行拟合，但前辈从场磨传感器的原理与构造本身给我讲了一下，让我知道了电压本身并不是唯一的输入变量，测试时的温度湿度，测量时电压的摆放方向，测量过程中的距离等等会影响输出电压结果。
2. 多向前辈请教，自己可能会钻牛角尖。在我为数据划分了测试集和测试集后，模型仍然产生过拟合，前辈指出可能是同一个电压的不同数据被分到了不同集合，导致虽然数据本身没有重叠，但是特征是重叠的，从而建议我按照电压分组后再分数据集，从而很大程度上解决了过拟合的现象。